# Convolutional Neural Networks

Emre Ozfatura[1]

[1]Intelligent Systems Lab.
Sabanci University, Turkey

4 March 2026

# Efficient CNNs

- CNNs are computationally less costly compared to fully connected layers.
- However, in deep architectures, as the number of channel increases, we need larger filters.
- For example, a filter with parameters $C_{in} = 256, C_{out} = 512, kh = 3, kw = 3$ have around $10^6$ number of weights.
- Here, we will discus how to reduce the computational complexity of the CNNs.

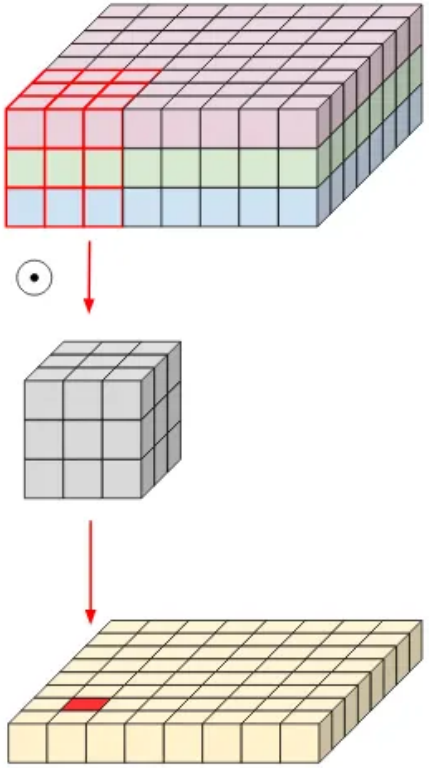

Figure 1: Illustration of standard convolution

**Notice:** For a single filter its 3D dot product followed by summation:

$$\mathbf{Y}[i,j] = \sum_{c=0}^{C_{in}-1} \sum_{m=0}^{kh-1} \sum_{n=0}^{kw-1} \mathbf{H}[c,m,n]\,\mathbf{X}[c,i,j]. \tag{1}$$

# Floating point operations (FLOPs)

- **FLOPs (Floating Point Operations)** measure the number of arithmetic operations performed by a neural network during a forward pass.
- Typical floating point operations include:
  - Addition: $a + b$
  - Subtraction: $a - b$
  - Multiplication: $a \times b$
  - Division: $a/b$
- In deep learning, the majority of FLOPs come from:
  - Convolution layers
  - Fully connected layers
  - Matrix multiplications
- For a convolution layer with: Input channels: $C_{in}$, Output channels: $C_{out}$, Kernel size: $K \times K$, Output feature map: $H_o \times W_o$. The number of FLOPs is:

$$\text{FLOPs} = 2 \times H_o \times W_o \times C_{out} \times C_{in} \times K^2$$

# Depth-wise convolution:

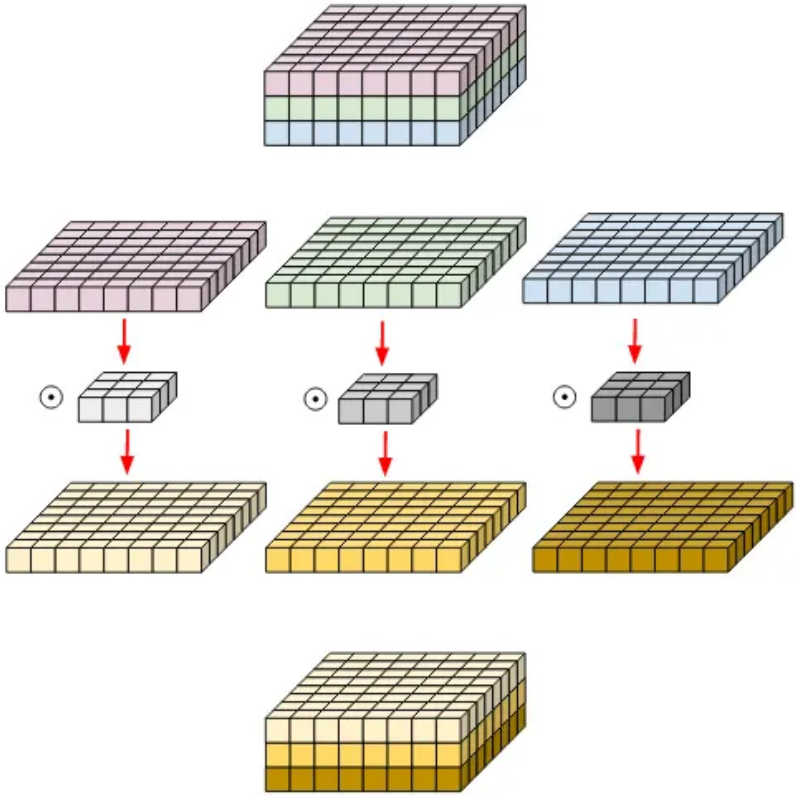

Figure 2: Illustration of depth-wise convolution

**Notice:** Each layer of the filter is used for a separate channel, then, the results are concatenated in the channel dimension.

# Group convolution

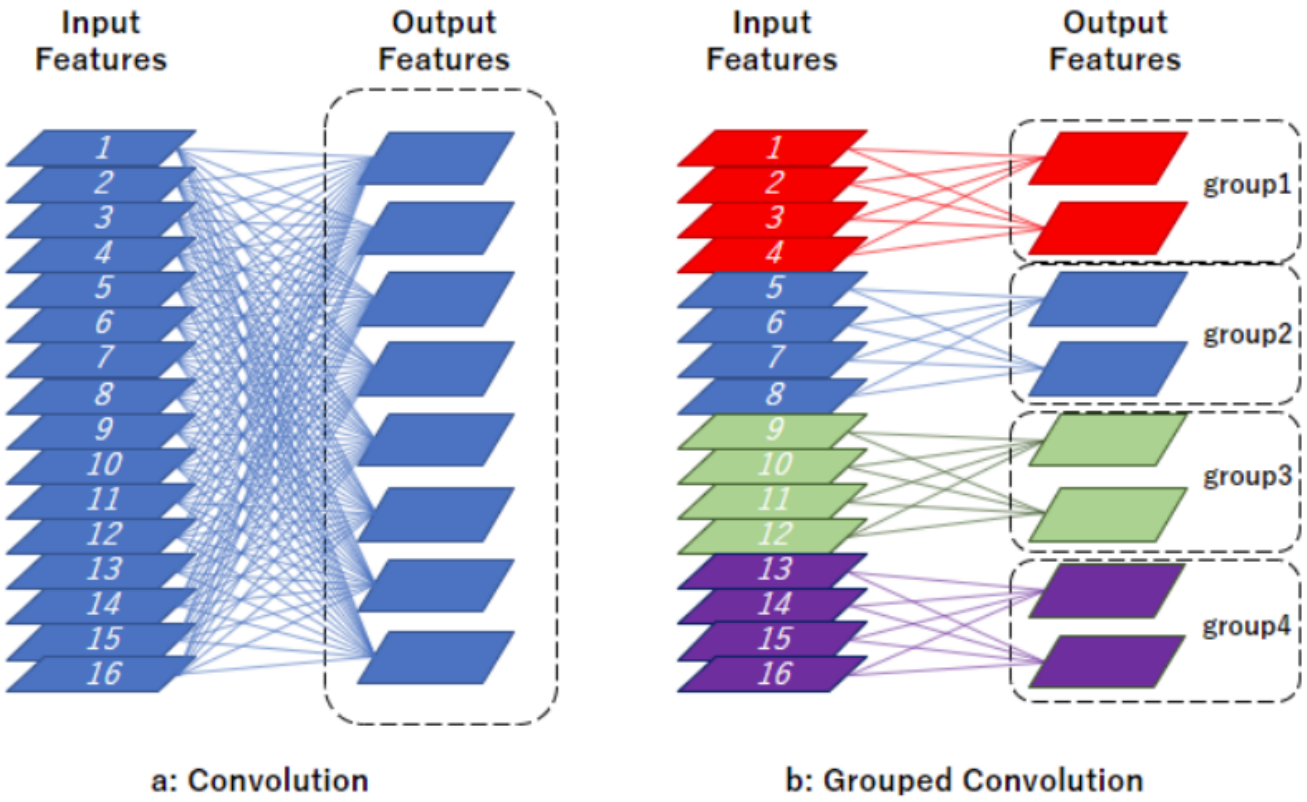


Figure 3: Illustration of group convolution.

# Group notion in Conv2D

**Shape:**

- Input: $(N, C_{in}, H_{in}, W_{in})$ or $(C_{in}, H_{in}, W_{in})$
- Output: $(N, C_{out}, H_{out}, W_{out})$ or $(C_{out}, H_{out}, W_{out})$, where

$$H_{out} = \left\lfloor \frac{H_{in} + 2 \times \text{padding}[0] - \text{dilation}[0] \times (\text{kernel_size}[0] - 1) - 1}{\text{stride}[0]} + 1 \right\rfloor$$

$$W_{out} = \left\lfloor \frac{W_{in} + 2 \times \text{padding}[1] - \text{dilation}[1] \times (\text{kernel_size}[1] - 1) - 1}{\text{stride}[1]} + 1 \right\rfloor$$

**Variables:**

- **weight** (*Tensor*) – the learnable weights of the module of shape $(\text{out_channels}, \frac{\text{in_channels}}{\text{groups}}, \text{kernel_size}[0], \text{kernel_size}[1])$. The values of these weights are sampled from $\mathcal{U}(-\sqrt{k}, \sqrt{k})$ where $k = \frac{groups}{C_{\text{in}} * \prod_{i=0}^{1} \text{kernel_size}[i]}$
- **bias** (*Tensor*) – the learnable bias of the module of shape (out_channels). If `bias` is `True`, then the values of these weights are sampled from $\mathcal{U}(-\sqrt{k}, \sqrt{k})$ where $k = \frac{groups}{C_{\text{in}} * \prod_{i=0}^{1} \text{kernel_size}[i]}$

**Notice:** By changing the number of group we can implement separable convolution.

**Watch:** `https://www.youtube.com/watch?v=vVaRhZXovbw`

# ResNet to ResNeXt [9]

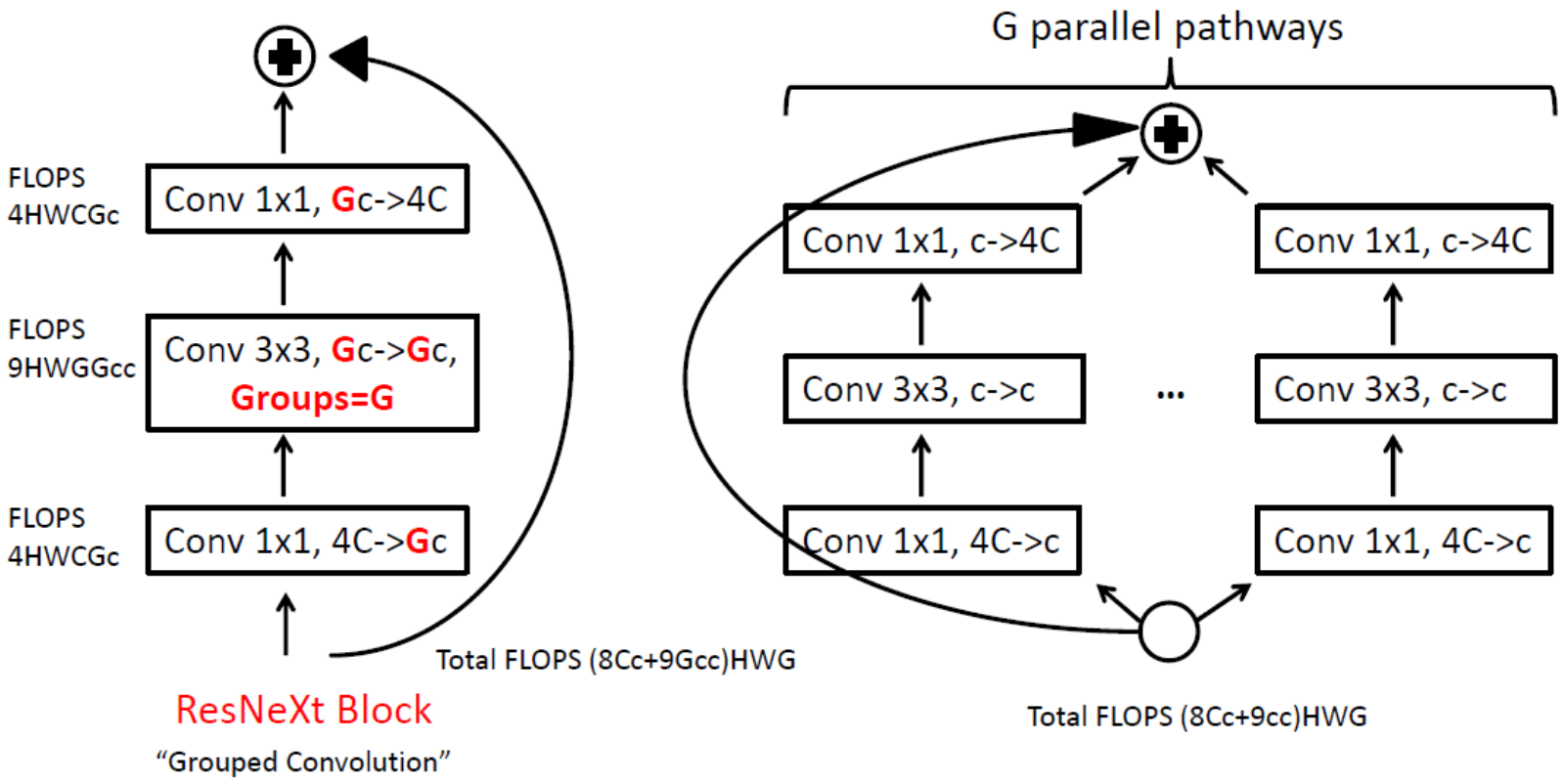


Figure 4: Illustration of group convolution.

# ResNeXt [9]

| stage | output | ResNet-50 | **ResNeXt-50 (32×4d)** |
|---|---|---|---|
| conv1 | 112×112 | 7×7, 64, stride 2 | 7×7, 64, stride 2 |
| conv2 | 56×56 | 3×3 max pool, stride 2 | 3×3 max pool, stride 2 |
| | | [ 1×1, 64<br>3×3, 64<br>1×1, 256 ] ×3 | [ 1×1, 128<br>3×3, 128, $C$=32<br>1×1, 256 ] ×3 |
| conv3 | 28×28 | [ 1×1, 128<br>3×3, 128<br>1×1, 512 ] ×4 | [ 1×1, 256<br>3×3, 256, $C$=32<br>1×1, 512 ] ×4 |
| conv4 | 14×14 | [ 1×1, 256<br>3×3, 256<br>1×1, 1024 ] ×6 | [ 1×1, 512<br>3×3, 512, $C$=32<br>1×1, 1024 ] ×6 |
| conv5 | 7×7 | [ 1×1, 512<br>3×3, 512<br>1×1, 2048 ] ×3 | [ 1×1, 1024<br>3×3, 1024, $C$=32<br>1×1, 2048 ] ×3 |
| | 1×1 | global average pool<br>1000-d fc, softmax | global average pool<br>1000-d fc, softmax |
| # params. | | $\mathbf{25.5}\times10^6$ | $\mathbf{25.0}\times10^6$ |
| FLOPs | | $\mathbf{4.1}\times10^9$ | $\mathbf{4.2}\times10^9$ |

Figure 5: Overall architecture.

# MobileNet [2]

Table 1. MobileNet Body Architecture

| Type / Stride | Filter Shape | Input Size |
|---|---|---|
| Conv / s2 | $3 \times 3 \times 3 \times 32$ | $224 \times 224 \times 3$ |
| Conv dw / s1 | $3 \times 3 \times 32$ dw | $112 \times 112 \times 32$ |
| Conv / s1 | $1 \times 1 \times 32 \times 64$ | $112 \times 112 \times 32$ |
| Conv dw / s2 | $3 \times 3 \times 64$ dw | $112 \times 112 \times 64$ |
| Conv / s1 | $1 \times 1 \times 64 \times 128$ | $56 \times 56 \times 64$ |
| Conv dw / s1 | $3 \times 3 \times 128$ dw | $56 \times 56 \times 128$ |
| Conv / s1 | $1 \times 1 \times 128 \times 128$ | $56 \times 56 \times 128$ |
| Conv dw / s2 | $3 \times 3 \times 128$ dw | $56 \times 56 \times 128$ |
| Conv / s1 | $1 \times 1 \times 128 \times 256$ | $28 \times 28 \times 128$ |
| Conv dw / s1 | $3 \times 3 \times 256$ dw | $28 \times 28 \times 256$ |
| Conv / s1 | $1 \times 1 \times 256 \times 256$ | $28 \times 28 \times 256$ |
| Conv dw / s2 | $3 \times 3 \times 256$ dw | $28 \times 28 \times 256$ |
| Conv / s1 | $1 \times 1 \times 256 \times 512$ | $14 \times 14 \times 256$ |
| $5\times$ Conv dw / s1 | $3 \times 3 \times 512$ dw | $14 \times 14 \times 512$ |
| Conv / s1 | $1 \times 1 \times 512 \times 512$ | $14 \times 14 \times 512$ |
| Conv dw / s2 | $3 \times 3 \times 512$ dw | $14 \times 14 \times 512$ |
| Conv / s1 | $1 \times 1 \times 512 \times 1024$ | $7 \times 7 \times 512$ |
| Conv dw / s2 | $3 \times 3 \times 1024$ dw | $7 \times 7 \times 1024$ |
| Conv / s1 | $1 \times 1 \times 1024 \times 1024$ | $7 \times 7 \times 1024$ |
| Avg Pool / s1 | Pool $7 \times 7$ | $7 \times 7 \times 1024$ |
| FC / s1 | $1024 \times 1000$ | $1 \times 1 \times 1024$ |
| Softmax / s1 | Classifier | $1 \times 1 \times 1000$ |

Figure 6: Overall architecture [2].

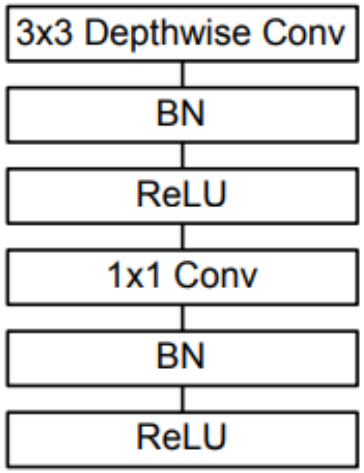


Figure 7: Underlying depth-wise separable convolution block.

# Bottleneck to inverted bottleneck

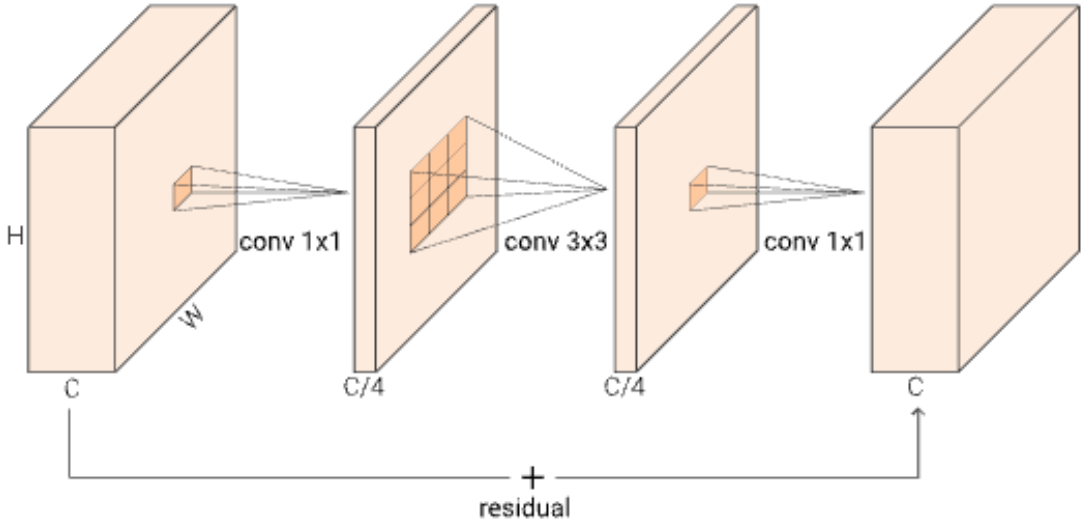


Figure 8: Illustration of bottleneck block.

**Recall:** We reduce the depth using point-wise convolution (Conv $1 \times 1$) to reduce the computation.

# Bottleneck to inverted bottleneck

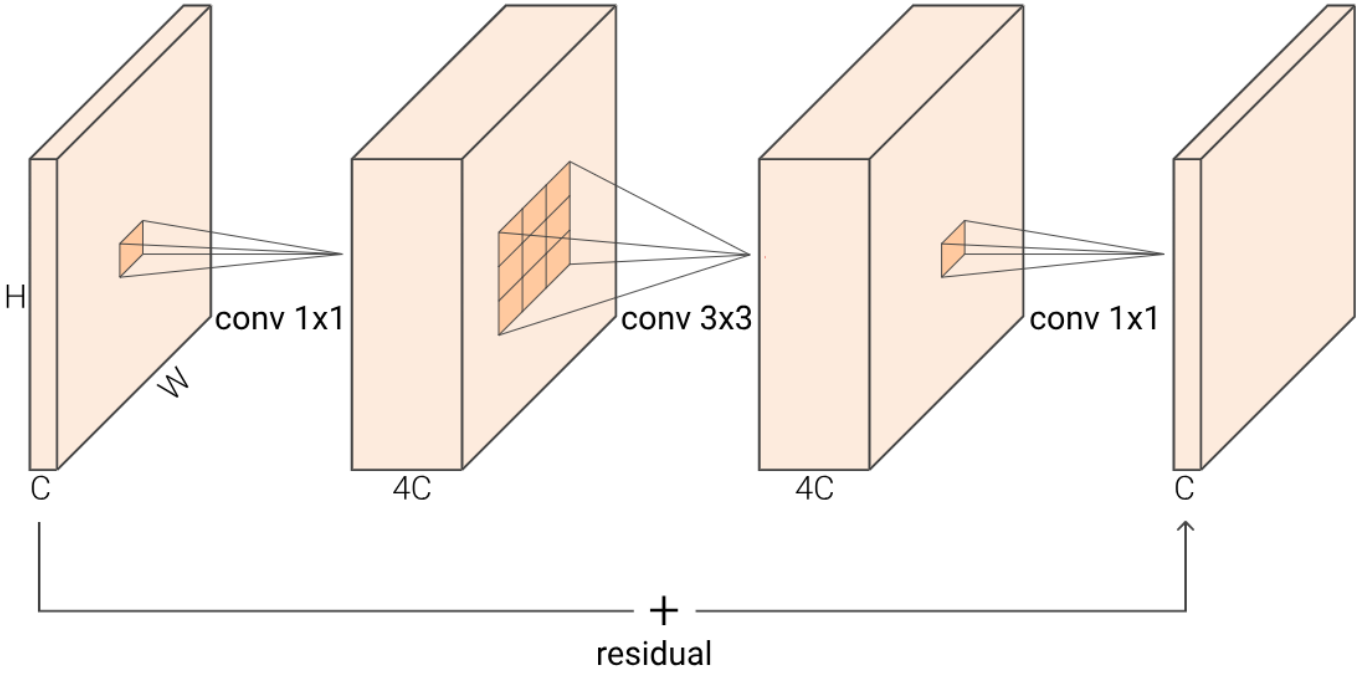


Figure 9: Illustration of inverted bottleneck block.

**Notice:** Now, we increase the number of channel using point-wise convolution (Conv $1 \times 1$). **WHY!!!**

# MobileNet Conv (MbConv) [6]

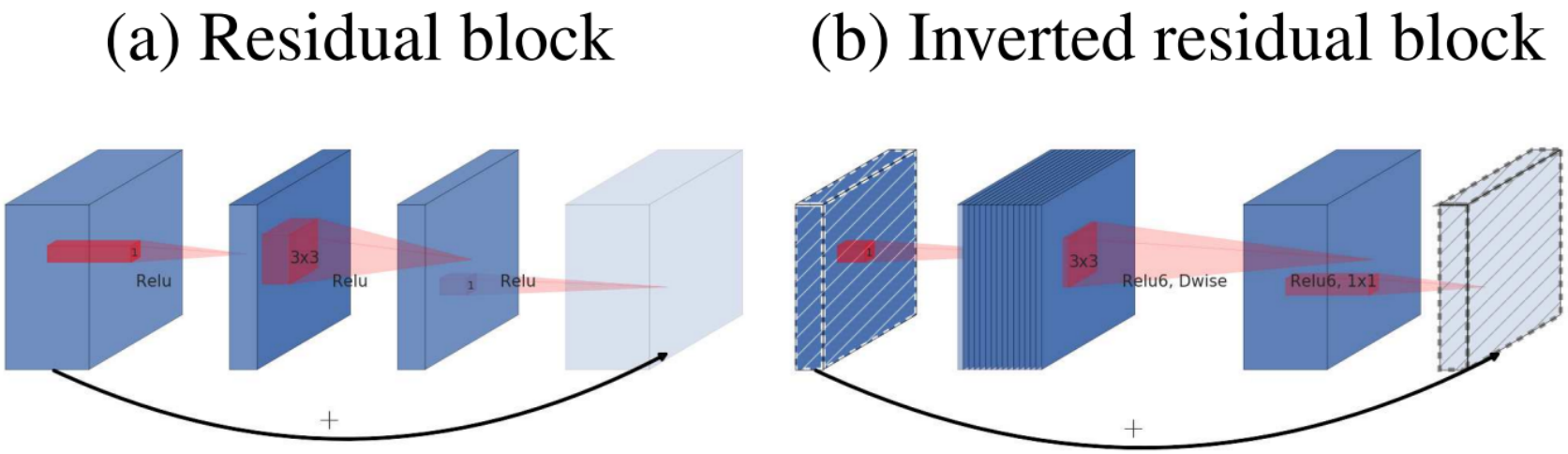


Figure 10: Inverted bottleneck/residual in MobileNet-V2 [6]

**Notice:** We increase the depth, however, use DwConv instead of Conv which reduces the complexity. **Note:** This inverted block also referred as MbConv block in the literature.

# MobileNet-V2

- Use inverted bottleneck with depth-wise convolution as main block
- Use ReLU6 $\mathrm{ReLU6}(x) = \min\big(\max(0, x), 6\big)$, for numerical stability.

| Input | Operator | $t$ | $c$ | $n$ | $s$ |
|---|---|---|---|---|---|
| $224^2 \times 3$ | conv2d | - | 32 | 1 | 2 |
| $112^2 \times 32$ | bottleneck | 1 | 16 | 1 | 1 |
| $112^2 \times 16$ | bottleneck | 6 | 24 | 2 | 2 |
| $56^2 \times 24$ | bottleneck | 6 | 32 | 3 | 2 |
| $28^2 \times 32$ | bottleneck | 6 | 64 | 4 | 2 |
| $14^2 \times 64$ | bottleneck | 6 | 96 | 3 | 1 |
| $14^2 \times 96$ | bottleneck | 6 | 160 | 3 | 2 |
| $7^2 \times 160$ | bottleneck | 6 | 320 | 1 | 1 |
| $7^2 \times 320$ | conv2d 1x1 | - | 1280 | 1 | 1 |
| $7^2 \times 1280$ | avgpool 7x7 | - | - | 1 | - |
| $1 \times 1 \times 1280$ | conv2d 1x1 | - | k | - | |

Figure 11: Overall architecture.

| Input | Operator | Output |
|---|---|---|
| $h \times w \times k$ | 1x1 conv2d , ReLU6 | $h \times w \times (tk)$ |
| $h \times w \times tk$ | 3x3 dwise s=$s$, ReLU6 | $\frac{h}{s} \times \frac{w}{s} \times (tk)$ |
| $\frac{h}{s} \times \frac{w}{s} \times tk$ | linear 1x1 conv2d | $\frac{h}{s} \times \frac{w}{s} \times k'$ |

Figure 12: Parameters of MbConv block.

# Squeeze-and-Excitation (SE) Block

- Let the input feature map be

$$\mathbf{X} \in \mathbb{R}^{C \times H \times W}.$$

- Global Average Pooling is applied to generate channel descriptors:

$$z_c = \frac{1}{HW} \sum_{i=1}^{H} \sum_{j=1}^{W} \mathbf{X}_c(i,j),$$

which produces

$$\mathbf{z} \in \mathbb{R}^{C}.$$

# Squeeze-and-Excitation (SE) Block

- The channel descriptor is passed through a bottleneck MLP:

$$\mathbf{s} = \sigma\left(\mathbf{W}_2\, \delta(\mathbf{W}_1 \mathbf{z})\right),$$

  where

$$\mathbf{W}_1 \in \mathbb{R}^{\frac{C}{r} \times C}, \quad \mathbf{W}_2 \in \mathbb{R}^{C \times \frac{C}{r}},$$

$$\delta(\cdot) = \text{ReLU}, \quad \sigma(\cdot) = \text{Sigmoid},$$

  and $r$ is the reduction ratio.

- The original feature map is re-calibrated channel-wise:

$$\tilde{\mathbf{X}}_c = s_c \cdot \mathbf{X}_c.$$

# Squeeze-and-Excitation (SE) Block

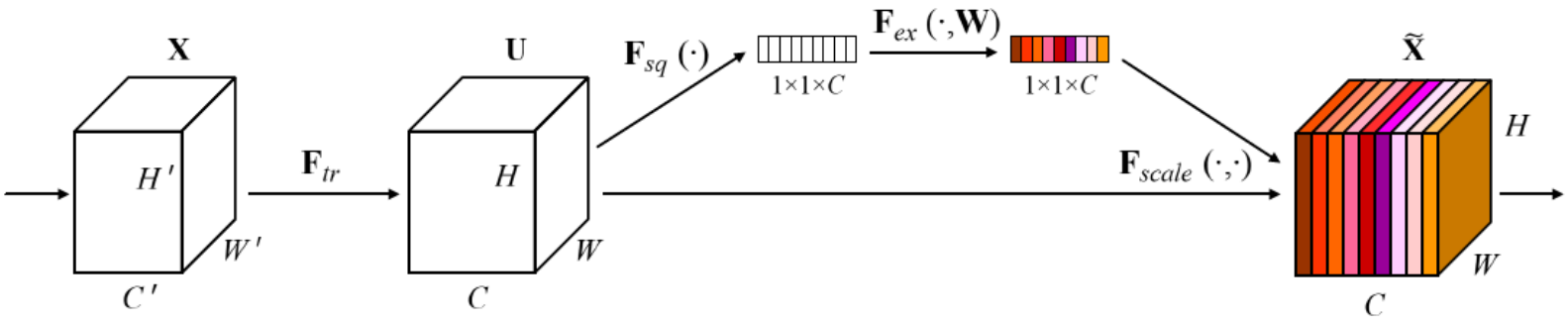


Figure 13: Illustration of Squeeze-and-Excitation (SE) Block (Hu et al. CVPR 2018 [3])

# Squeeze-and-Excitation (SE) Block

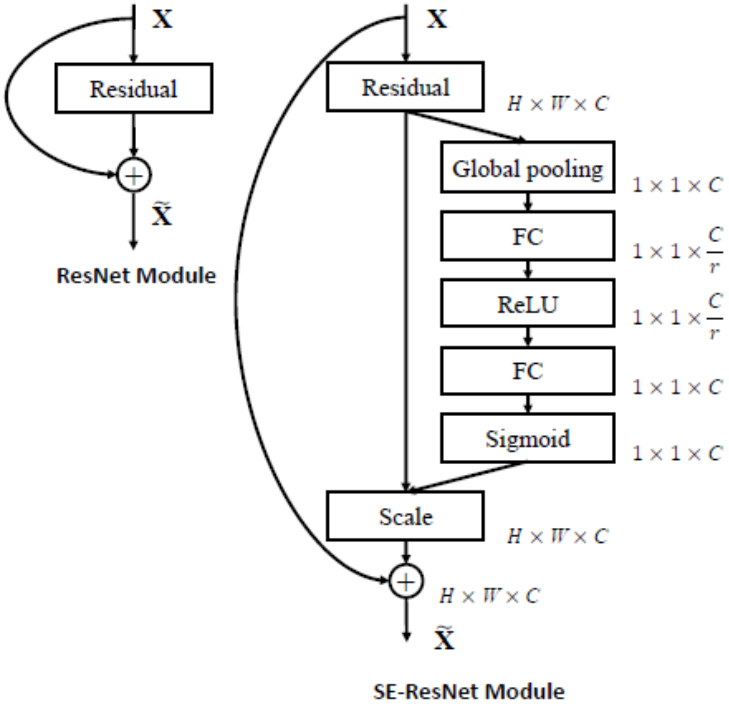


Figure 14: Illustration of Squeeze-and-Excitation (SE) Block in ResNet

# Squeeze-and-Excitation (SE) Block with neighboring connection

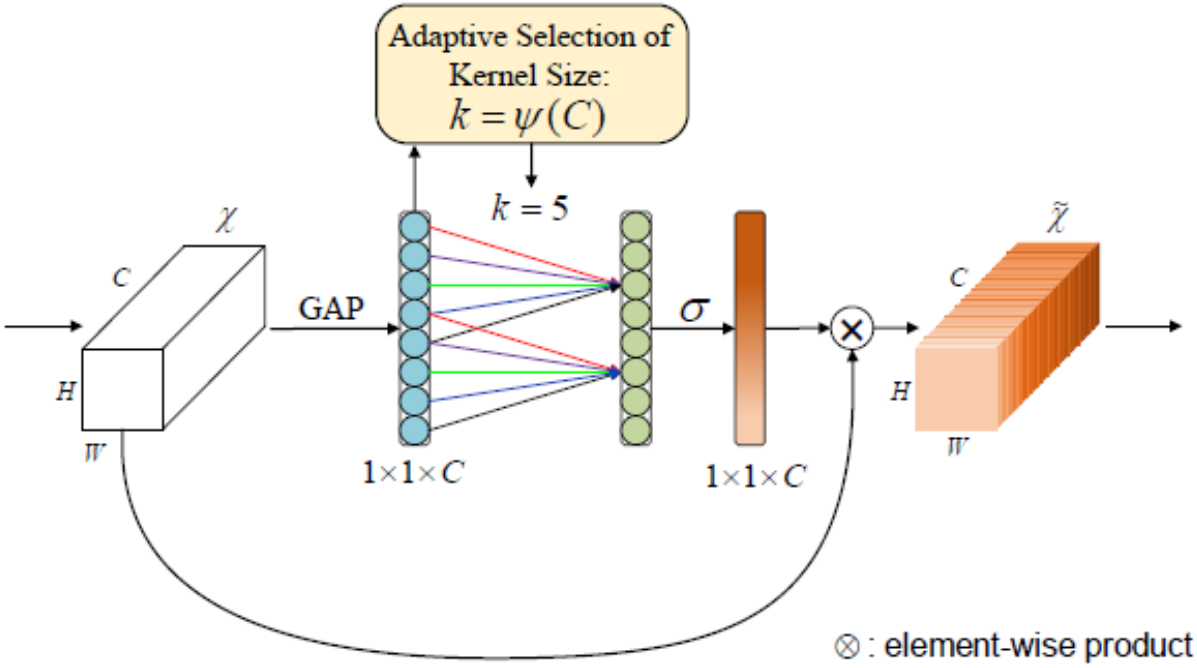


Figure 15: Illustration of efficient implementation of Squeeze-and-Excitation (SE) Wang et al. CVPR 2020

**Notice:** Instead of using fully connected layer, we consider neighboring $k$ channels.

# Shuffle-Net

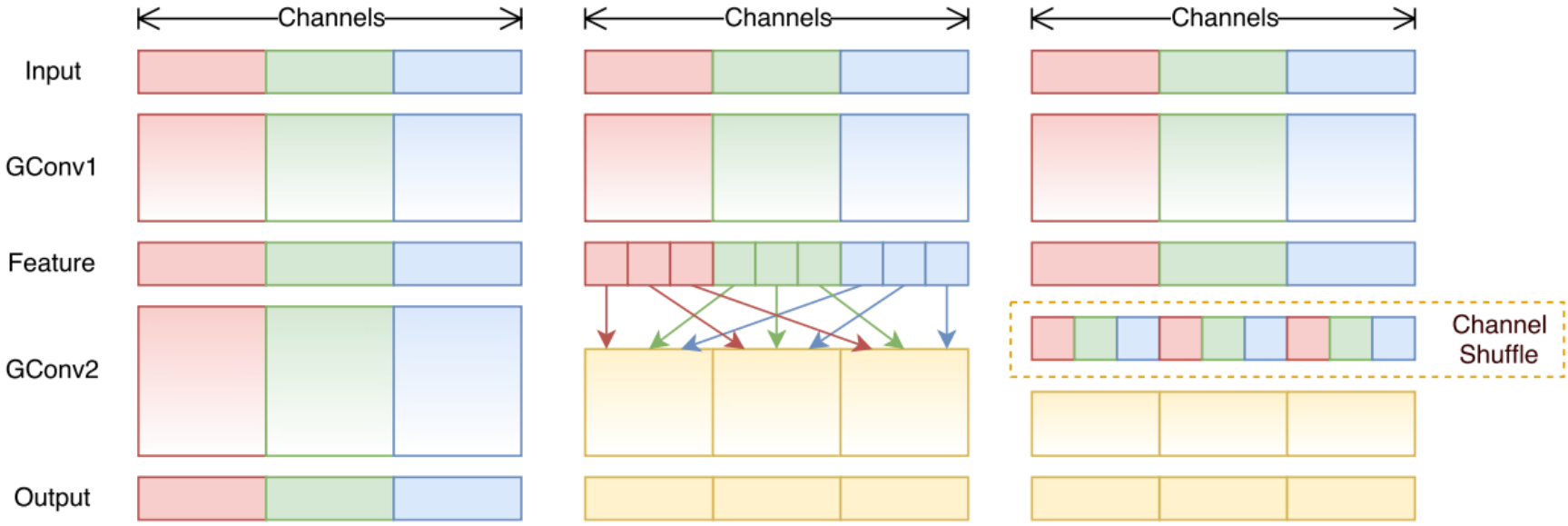


Figure 16: Message passing through channel shuffling (Zhang et al. CVPR 2018.)

**Notice:** Group convolution reduce the complexity, but we need a message passing between different channel groups, hence employ a channel shuffling framework.

# Shuffle-Net

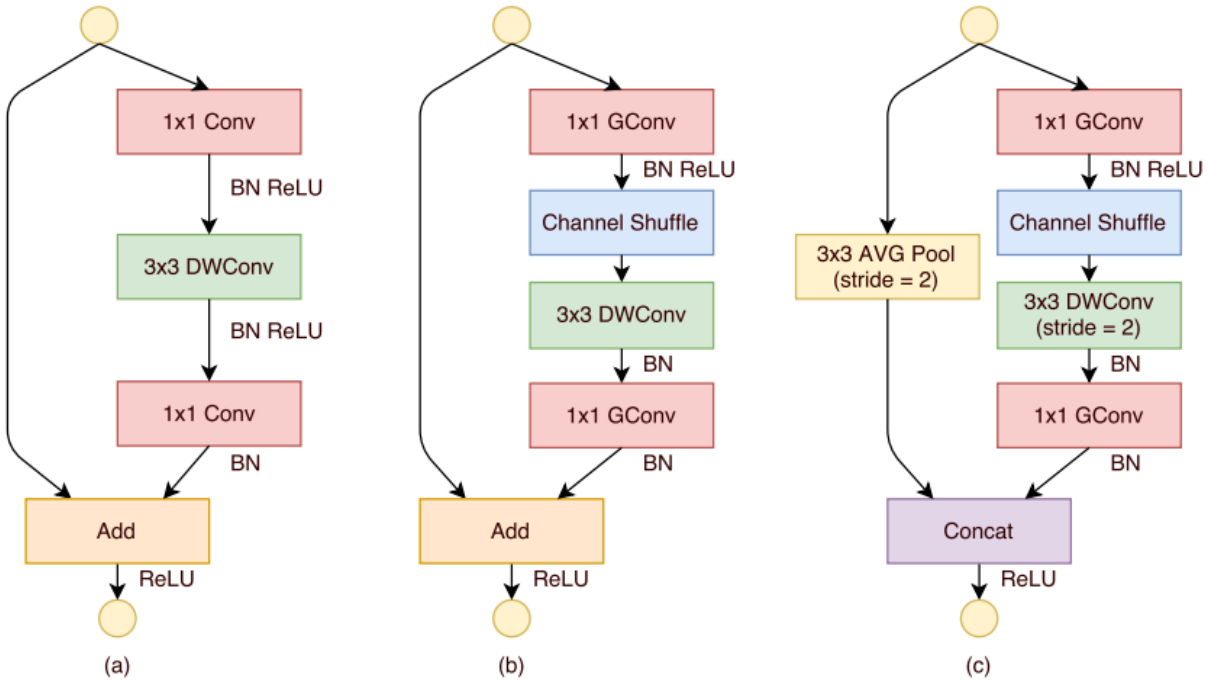


Figure 17: ShuffleNet block (Zhang et al. CVPR 2018. [10])

**b)**ShuffleNet unit with pointwise group convolution (GConv) and channel shuffle; **c)** ShuffleNet unit with stride $= 2$. **Note:** For GConv we set number of groups equal to $G$, having $C_{in}/G$ channel per group, for DWConv we set number of groups equal to $C_{in}$.

# Shuffle-Netv2

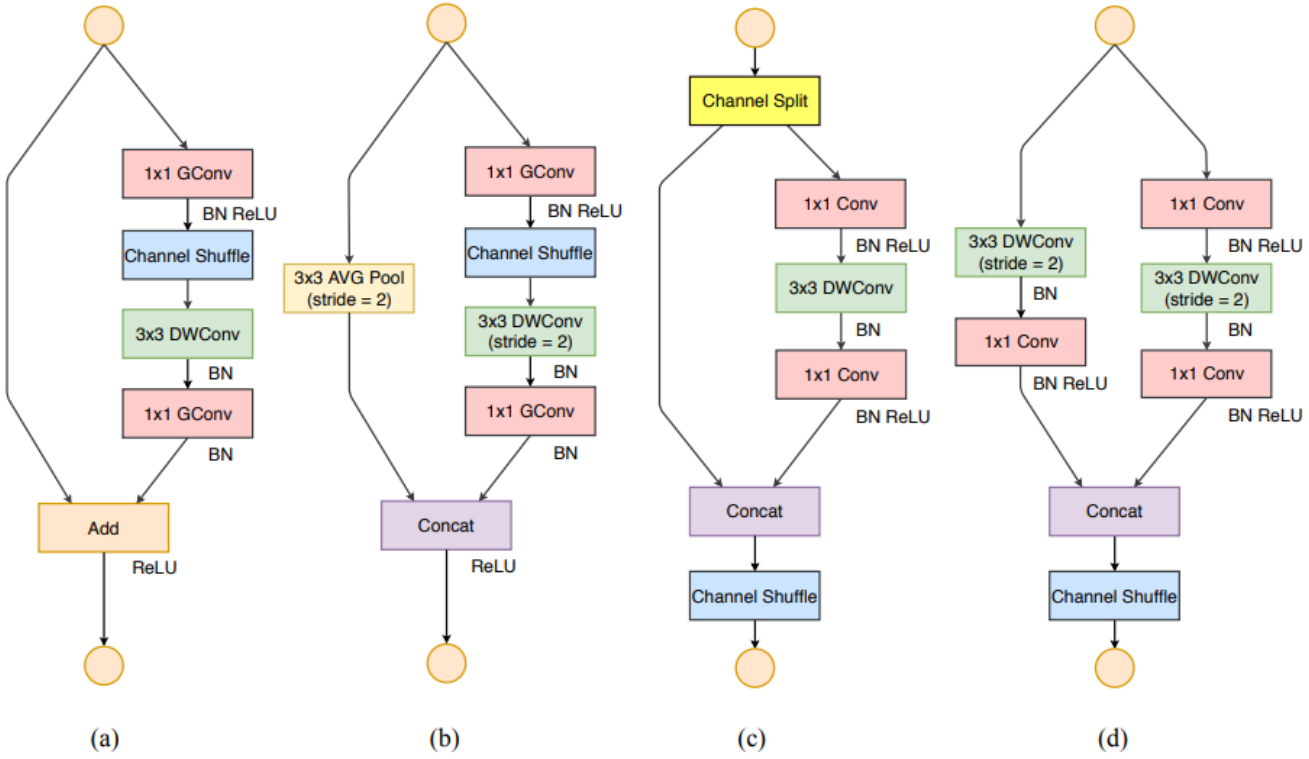


Figure 18: Comparison of Shuffle-Net with Shuffle-Netv2 [4].

**(a)**: the basic ShuffleNet unit; **(b)** the ShuffleNet unit for spatial down sampling (2×); **(c)** proposed basic unit; **(d)** proposed unit for spatial down sampling (2×).

# Neural architecture search

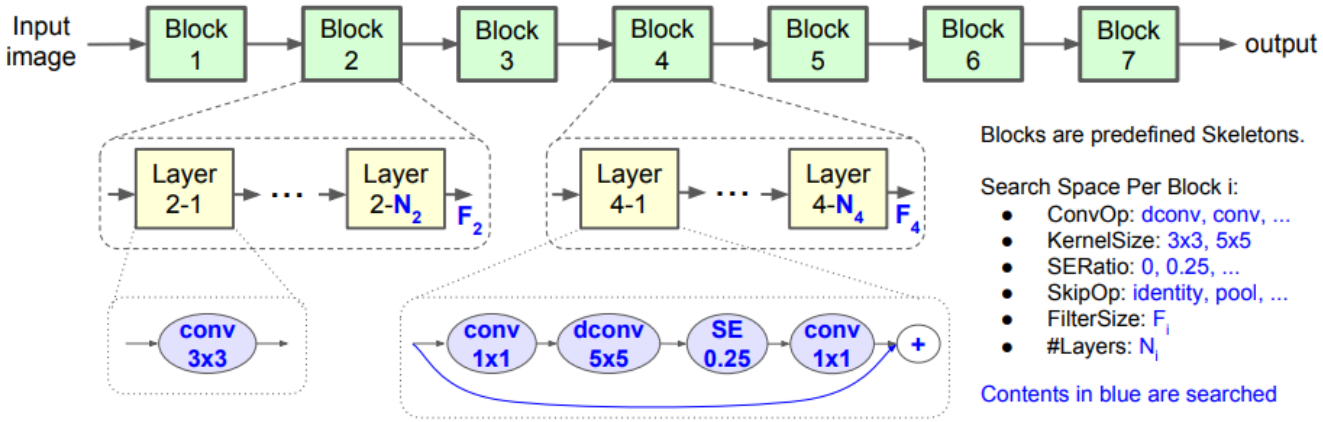


Figure 19: MnasNet search space [7].

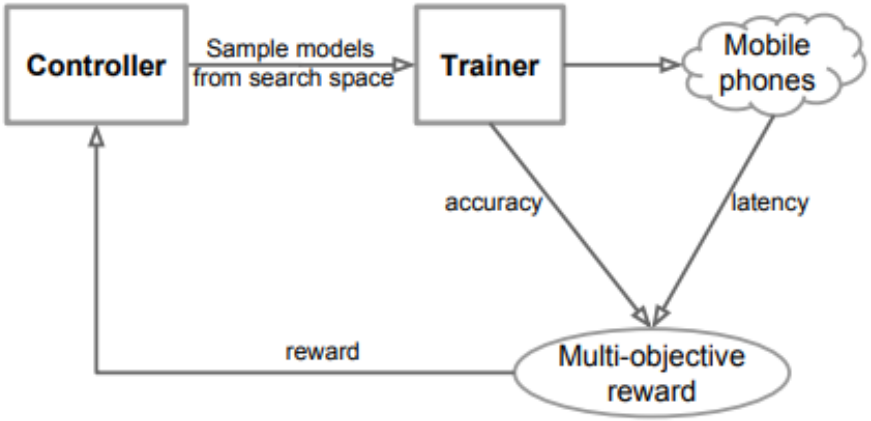


Figure 20: Search framework

# EfficientNet [8]

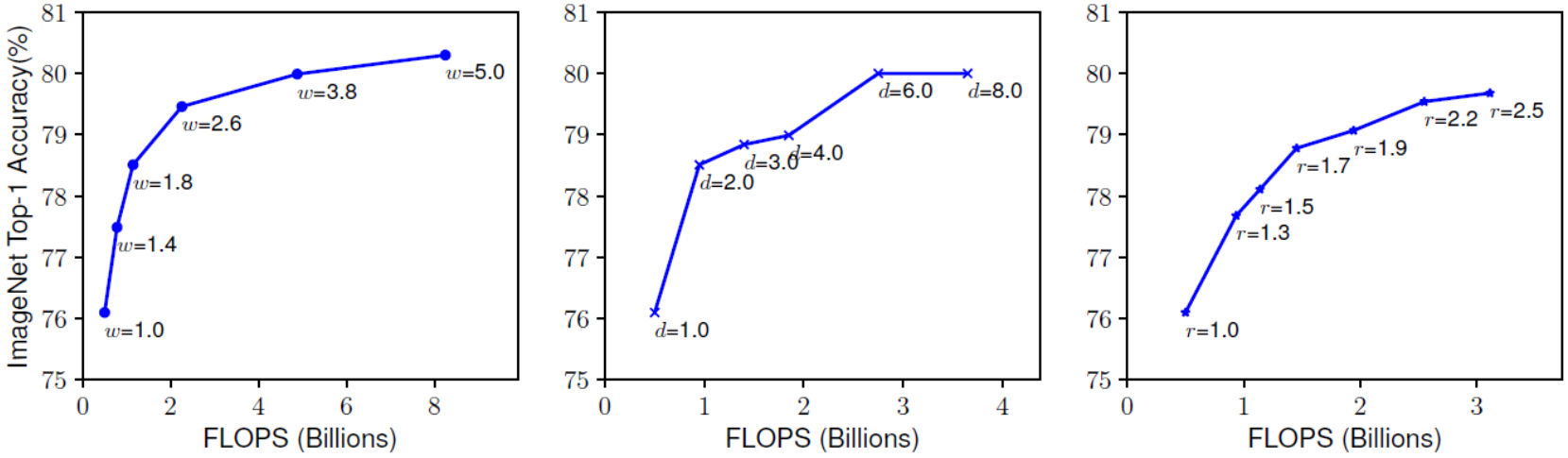


Figure 21: Scaling Up a Baseline Model with Different Network Width (w), Depth (d), and Resolution (r) Coefficients [8]

**Notice:** Scaling the network only in a single direction has a diminishing return.

# EfficientNet (Compound scaling)

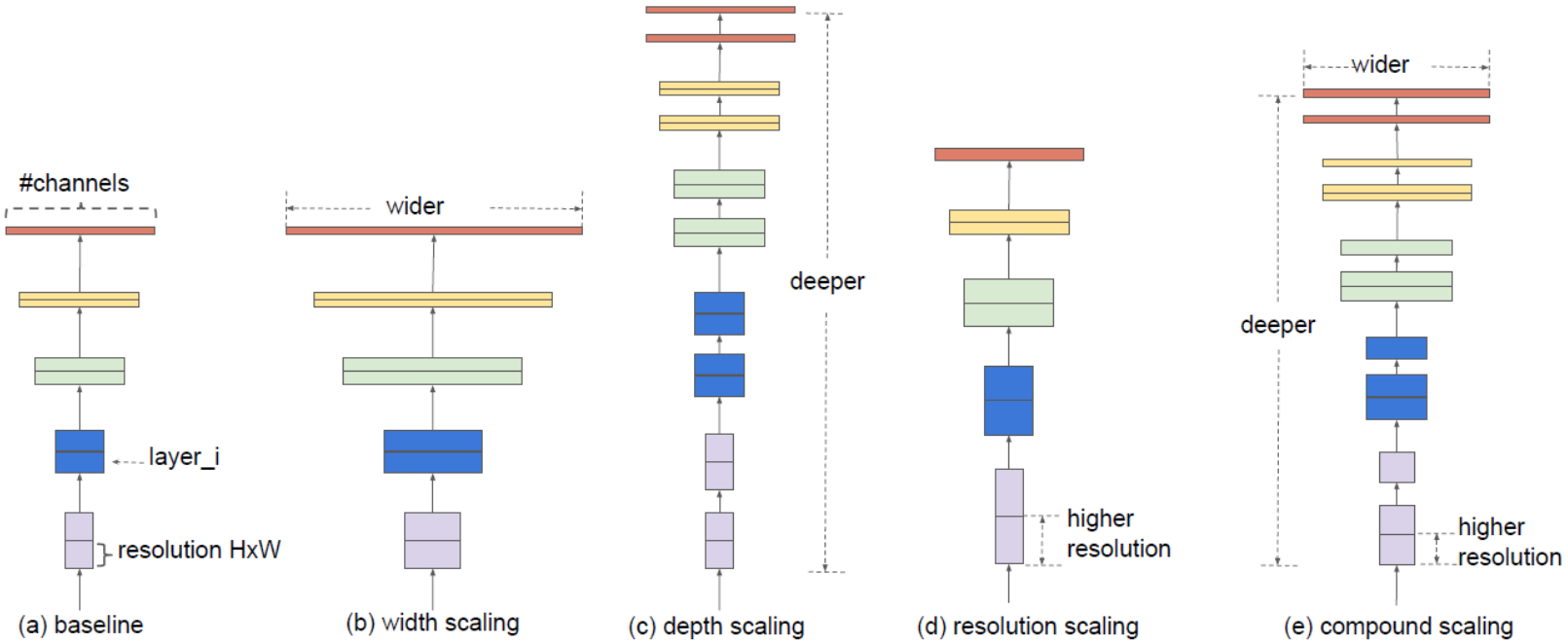


Figure 22: Compound scaling in EfficientNet [8]

**Remark:** Scaling the network only at single direction does not bring

# EfficientNet (Compound scaling)

A single scaling coefficient $\phi$ controls the scaling:

$$d = \alpha^{\phi}$$

$$w = \beta^{\phi}$$

$$r = \gamma^{\phi}$$

where

- $d$ : depth scaling
- $w$ : width scaling
- $r$ : resolution scaling
- $\alpha, \beta, \gamma$ : constants determined by grid search
- $\phi$ : compound scaling coefficient

The scaling factors satisfy the constraint:

$$\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$$

# EfficientNet (Backbone)

| Stage $i$ | Operator $\hat{\mathcal{F}}_i$ | Resolution $\hat{H}_i \times \hat{W}_i$ | #Channels $\hat{C}_i$ | #Layers $\hat{L}_i$ |
|---|---|---|---|---|
| 1 | Conv3x3 | $224 \times 224$ | 32 | 1 |
| 2 | MBConv1, k3x3 | $112 \times 112$ | 16 | 1 |
| 3 | MBConv6, k3x3 | $112 \times 112$ | 24 | 2 |
| 4 | MBConv6, k5x5 | $56 \times 56$ | 40 | 2 |
| 5 | MBConv6, k3x3 | $28 \times 28$ | 80 | 3 |
| 6 | MBConv6, k5x5 | $14 \times 14$ | 112 | 3 |
| 7 | MBConv6, k5x5 | $14 \times 14$ | 192 | 4 |
| 8 | MBConv6, k3x3 | $7 \times 7$ | 320 | 1 |
| 9 | Conv1x1 & Pooling & FC | $7 \times 7$ | 1280 | 1 |

Figure 23: Compound scaling in EfficientNet [8]

**Notice:** EfficientNet build on MBConv blocks.

# EficientNet

1. **Step 1.** Use NAS to obtain the initial baseline EfficientNet-B0 architecture; it uses depthwise convolutions, inverted bottlenecks, and SE blocks.
2. **Step 2.** Find the optimal scaling factors $\alpha$ for depth, $\beta$ for width, and $\gamma$ for resolution, with

$$\alpha\beta\gamma \geq 1 \quad \text{and} \quad \alpha\beta^2\gamma^2 \approx 2$$

   via grid search by scaling up the baseline architecture. The resulting values are:

$$\alpha = 1.2, \quad \beta = 1.1, \quad \gamma = 1.15$$

3. **Step 3.** Scale the initial architecture to arbitrary FLOPs. Scaling by $\alpha^\phi$, $\beta^\phi$, and $\gamma^\phi$ increases the FLOPs by approximately

$$2^\phi.$$

# MobileNet-V3 [1]

- Employ a platform-aware neural architecture approach to find the global network structures (reuse the same MnasNet-A1 [7])
- Follow the MobileNet legacy, but also employ the notion of squeeze and excite.
- Use modified version of swish

$$h - swish(x) = x\frac{ReLU6(X+3)}{6} \tag{2}$$

instead of

$$swish(x) = x\sigma(x), \tag{3}$$

where $\sigma(x)$ is sigmoid function

# RegNet [5]

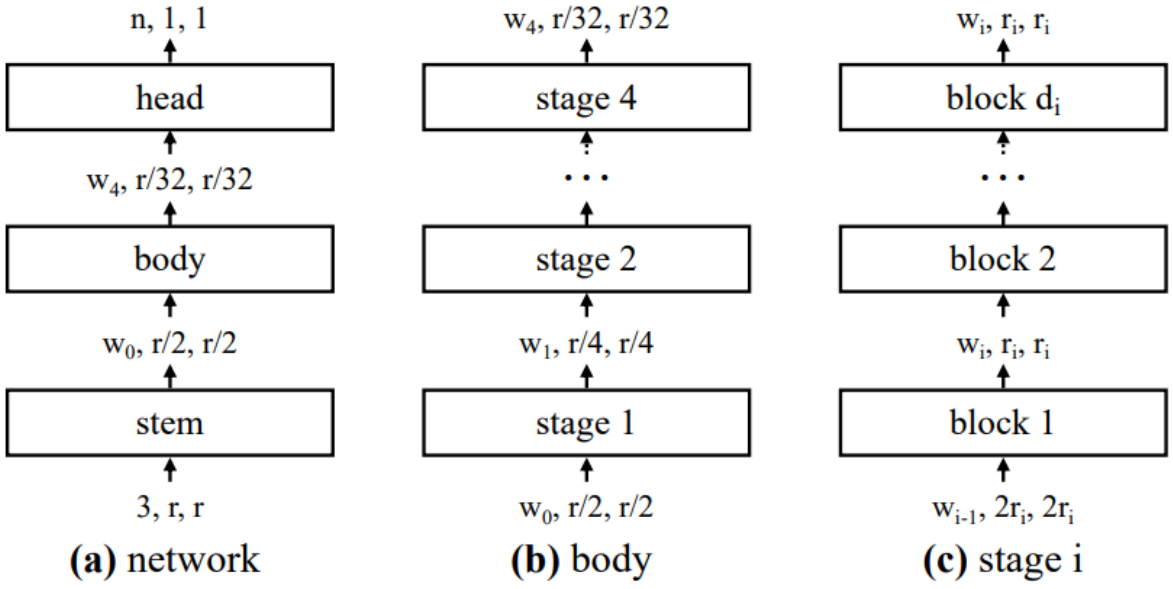


Figure 24: RegNet design space

**General design spaces:** (a) Each network consists of a *stem* (stride-two $3 \times 3$ conv with $w_0 = 32$ output channels), followed by the network body and then a *head* (average pooling followed by a fully connected layer) that predicts $n$ output classes.
(b) The network body is composed of a sequence of stages that operate at progressively reduced resolution $r_i$.
(c) Each stage consists of a sequence of identical blocks.

# RegNet [5]

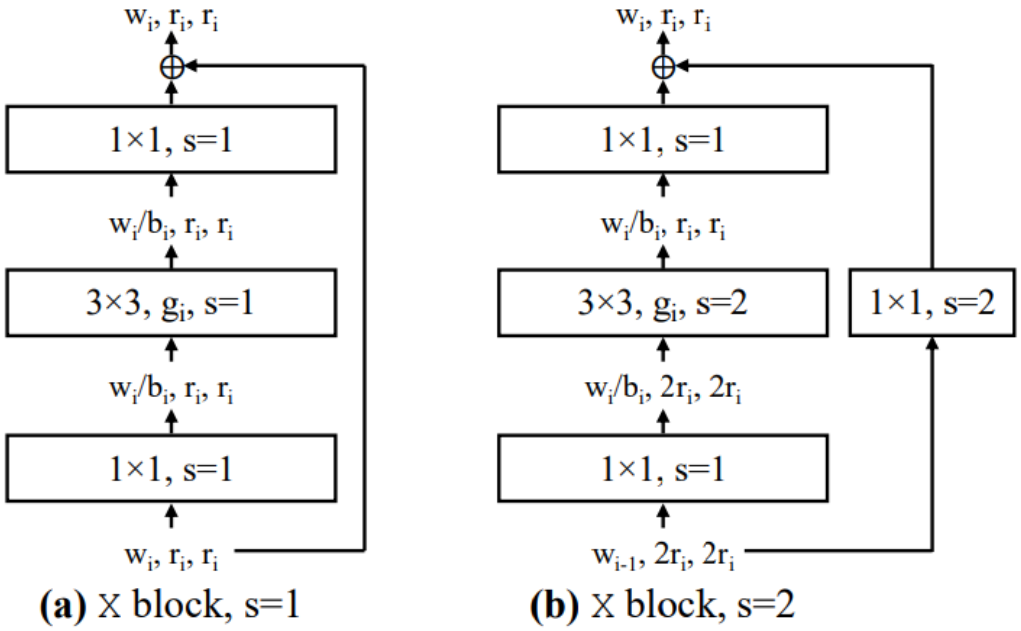


Figure 25: RegNet Blocks

The X block is based on the standard residual bottleneck block with group convolution (a) Each X block consists of a $1 \times 1$ conv, a $3 \times 3$ group conv, and a final $1 \times 1$ conv, where the $1 \times 1$ convs alter the channel width. BatchNorm and ReLU follow each conv. The block has three parameters: the width $w_i$, bottleneck ratio $b_i$, and group width $g_i$. (b) The stride-two ($s = 2$) version.

# References


- https://visionbook.mit.edu/convolutional_neural_nets.html
- https://github.com/FrancescoSaverioZuppichini/BottleNeck-InvertedResidual-FusedMBConv-in-PyTorch
- https://medium.com/@zurister/depth-wise-convolution-and-depth-wise-separable-convol
- https://www.youtube.com/watch?v=vVaRhZXovbw
- https://tvm.d2l.ai/chapter_common_operators/depthwise_conv.html

# Reference papers I


Andrew Howard, Mark Sandler, Grace Chu, Liang-Chieh Chen, Bo Chen, Mingxing Tan, Weijun Wang, Yukun Zhu, Ruoming Pang, Vijay Vasudevan, et al.
Searching for mobilenetv3.
In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 1314–1324, 2019.

Andrew G Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam.
Mobilenets: Efficient convolutional neural networks for mobile vision applications.
*arXiv preprint arXiv:1704.04861*, 2017.

# Reference papers II


Jie Hu, Li Shen, and Gang Sun.
Squeeze-and-excitation networks.
In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 7132–7141, 2018.

Ningning Ma, Xiangyu Zhang, Hai-Tao Zheng, and Jian Sun.
Shufflenet v2: Practical guidelines for efficient cnn architecture design.
In *Proceedings of the European Conference on Computer Vision (ECCV)*, September 2018.

Ilija Radosavovic, Raj Prateek Kosaraju, Ross Girshick, Kaiming He, and Piotr Dollar.
Designing network design spaces.
In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2020.

# Reference papers III


Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen.
Mobilenetv2: Inverted residuals and linear bottlenecks.
In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 4510–4520, 2018.

Mingxing Tan, Bo Chen, Ruoming Pang, Vijay Vasudevan, Mark Sandler, Andrew Howard, and Quoc V Le.
Mnasnet: Platform-aware neural architecture search for mobile.
In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 2820–2828, 2019.

Mingxing Tan and Quoc Le.
Efficientnet: Rethinking model scaling for convolutional neural networks.
In *International conference on machine learning*, pages 6105–6114. PMLR, 2019.

# Reference papers IV


Saining Xie, Ross Girshick, Piotr Dollar, Zhuowen Tu, and Kaiming He.
Aggregated residual transformations for deep neural networks.
In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, July 2017.

Xiangyu Zhang, Xinyu Zhou, Mengxiao Lin, and Jian Sun.
Shufflenet: An extremely efficient convolutional neural network for mobile devices.
In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 6848–6856, 2018.